

Development of a Low-Cost Computer Vision-Based Four-Wheeled Robot for Autonomous Navigation Using ESP32 Image Processing and Arduino Motor Control

Abhiram Kaakarla

Central Bucks High School South, Warrington, PA

Mentor: Dr. Vinayak Elangovan, Ph.D., *Program Chair of Computer Science,
Associate Professor of Computer Science*

Penn State Abington University, Abington, PA

ABSTRACT

This project explores the combination of computer vision, image processing, and foundational machine learning to create an autonomous navigation system for the Freenove 4WD robot car. Using an ESP32 camera module, the system captures live video frames that are processed in real time through a Python-based control system. The camera's visual input is converted to grayscale to reduce complexity while preserving essential features for edge detection and path recognition. Individual pixel-level data is analyzed to detect contours, lines, directional signs, and nearby obstacles using the OpenCV library. In addition to classical image processing techniques such as Canny edge detection, Hough line transformation, and color thresholding, the system incorporates basic machine learning classifiers trained on visual patterns. These models enable the robot to recognize directional changes such as left and right turns, T-intersections, and stop signs – allowing it to respond intelligently to environmental changes. Control decisions such as adjusting steering angles or changing speed are determined dynamically based on this analysis, creating real-time feedback between perception and motion. By simulating real-world autonomous vehicle behavior using low-cost hardware and accessible software, this project mirrors core functions found in commercial self-driving technologies, such as those used by Tesla. These vehicles rely heavily on camera-based perception systems, pixel analysis, and learned behavior models to safely navigate roads. The Freenove system mimics this by processing visual data to make navigation decisions, demonstrating how even small-scale robotics can contribute to a larger understanding of self-directed mobility. Ultimately, this project aims to make the principles behind advanced vehicle autonomy more approachable and applicable for learners and developers through hands-on implementation.

INTRODUCTION

Autonomous vehicle technology represents one of the most groundbreaking advancements in both robotics and artificial intelligence, with applications ranging from self-driving cars to warehouse robots and delivery drones. Major companies such as Tesla and Waymo rely on camera-based perception, deep neural networks, and real-time decision-making systems to create vehicles that can safely interpret and navigate complex environments. At the heart of these systems lies computer vision, an interdisciplinary field that enables machines to process and make sense of visual information, and machine learning, which allows systems to adapt to new patterns and learn from past experiences. This project seeks to explore how these same technologies can be applied on a more accessible, small-scale robotic platform such as the Freenove 4WD robot car. The robot uses an ESP32 camera module mounted to capture live video feed as it moves. This video data is continuously processed using Python and the OpenCV library to extract features like lane markings, obstacles, and stop signs. One of the initial steps involves converting the RGB video frames into grayscale to simplify the computation and highlight patterns within the scene, such as edges and contours. Pixel configuration becomes especially important here, as grayscale images allow for easier color thresholding and detection of line boundaries.

To simulate intelligent navigation, traditional image processing techniques are combined with simple machine learning models. These models are trained on visual datasets to recognize various driving scenarios such as when to turn, stop, or adjust speed. The robot can identify common cues, like the presence of a stop sign or a bend in the path, and respond by executing programmed actions such as braking, accelerating, or steering. This mimics how real autonomous vehicles use processed visual input to monitor real-time motor control, effectively connecting the areas of perception and movement. Furthermore, this system offers an educational opportunity to understand the layered structure of modern autonomy: from raw image capture to processed pixel data, to classification and decision-making. While Tesla and similar systems operate with highly complex neural networks and sensor fusion (integrating radar, LiDAR, etc.), this project illustrates a fundamental working model using only visual input and accessible software frameworks. It emphasizes how even basic machine learning can improve a robot's ability to respond to dynamic environments, which is key for real-world deployment. Ultimately, this work not only simulates the functionality of modern autonomous systems but also provides insight into how robotic perception, adaptive control, and image interpretation work together. It serves as both a practical learning tool and a large step towards developing more advanced autonomous agents, capable of reacting to their environment through both logic and learned behavior.

METHODS

Research Design

This project followed an experimental, systems-based research design focused on evaluating the feasibility of camera-based autonomous navigation using low-cost hardware. The robot's behavior was observed under controlled indoor conditions to assess how effectively image processing and simple machine learning techniques could guide navigation decisions. Multiple test environments were created, including straight paths, turns, intersections, and obstacle placements, to evaluate system performance under varying visual conditions.

Materials and Hardware

The primary platform used was the Freenove 4WD robot car, powered by four DC motors and controlled through an Arduino-based motor driver. An ESP32 camera module was mounted at the front of the robot to capture forward-facing visual input. Power was supplied by onboard rechargeable batteries. Serial communication between the ESP32 and Arduino enabled real-time transmission of navigation commands derived from visual analysis. Power was supplied through rechargeable lithium battery packs mounted beneath the chassis, providing regulated voltage to both the Arduino motor control subsystem and the ESP32 processing module. Voltage stability was critical to ensure consistent motor torque output and prevent brownout conditions during simultaneous motor actuation and image processing. Communication between the ESP32 and the Arduino microcontroller was established via UART serial protocol. The ESP32 transmitted structured command packets representing navigation decisions derived from visual processing. These packets encoded motor control parameters such as speed, turning direction, and stopping intervals. Upon receipt, the Arduino interpreted these signals and executed real-time motor

adjustments through the motor driver shield. This distributed hardware architecture separated perception (ESP32) from actuation (Arduino), improving processing efficiency and system modularity.

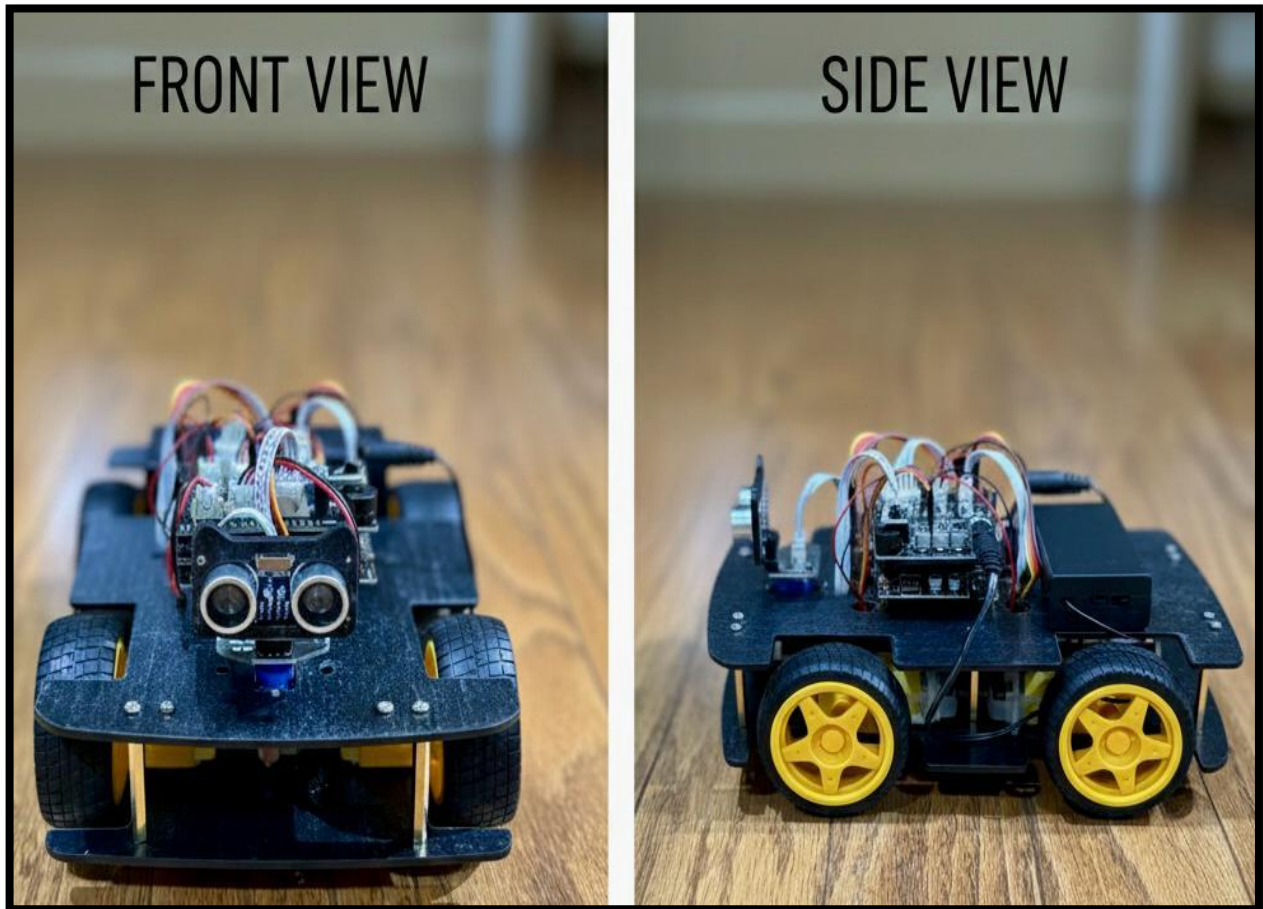


Figure 1: Low-Cost Autonomous Freenove 4WD Robot Front and Side View

Image Acquisition and Preprocessing

The ESP32 camera continuously captured grayscale image frames at reduced resolution to minimize memory usage and computational load. Grayscale conversion simplified pixel-level analysis while preserving essential structural features such as edges and contours. Each frame was resized prior to processing to ensure compatibility with the ESP32's memory constraints and to reduce processing latency overall for faster and efficient object detection and avoidance. The raw images captured by the ESP32 camera were fed to an edge impulse neural network model.

Image Processing and Recognition

Classical computer vision algorithms were applied to extract navigational features from each frame. Edge detection techniques highlighted lane boundaries and object outlines, while line detection assisted in determining path direction. Template-based image matching was implemented using preloaded grayscale reference images stored as header files. A sum-of-

absolute-differences (SAD) metric was used to quantify similarity between live frames and template images, enabling object classification.

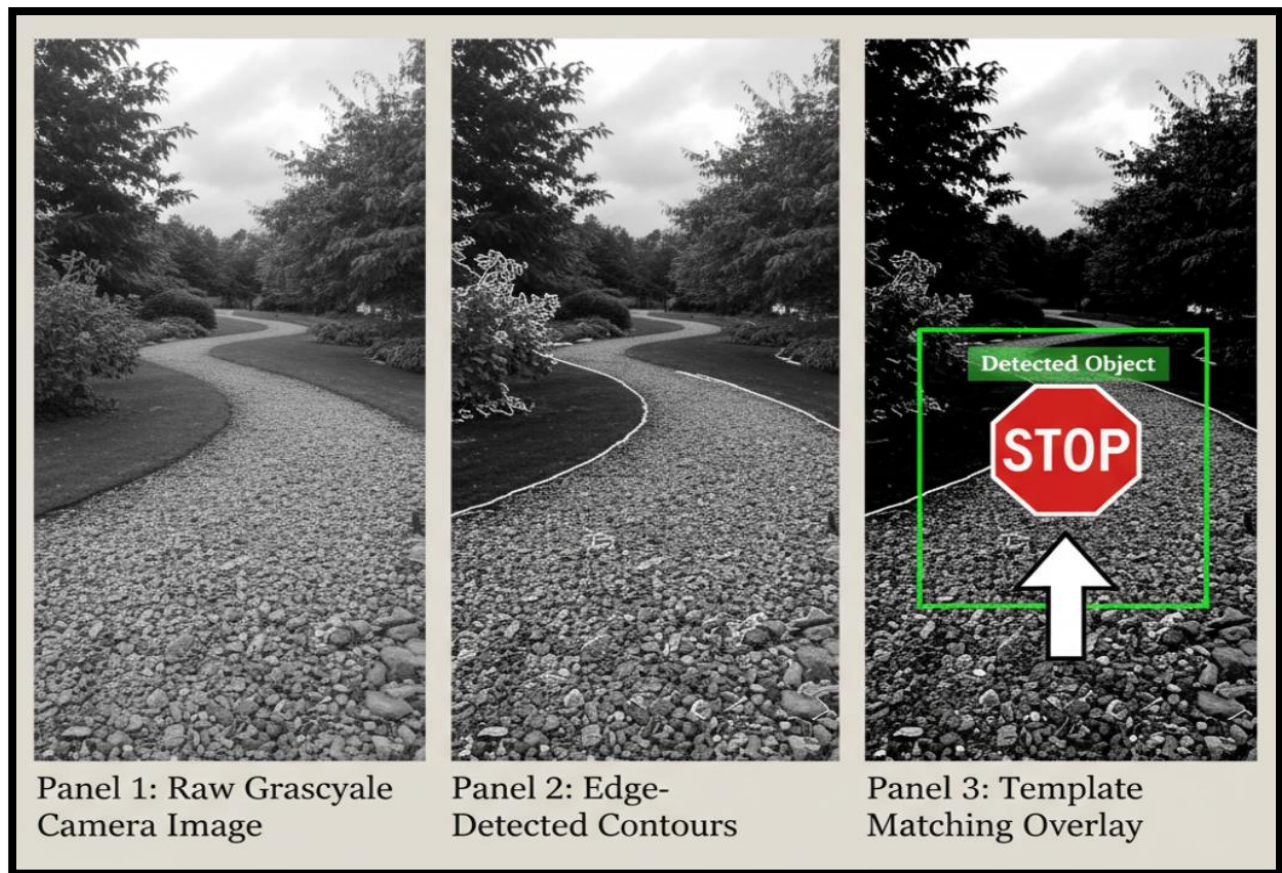


Figure 2: Image Processing Pipeline Example

Navigation and Control Logic

Once an object or directional cue was recognized through onboard image processing and neural network inference, the ESP32 generated the appropriate control decision and transmitted corresponding commands to the Arduino motor controller via serial communication. These commands specified adjustments to motor speed, steering direction, or braking behavior depending on the classified visual input. For example, detection of a stop sign triggered a temporary halt, while recognition of left or right turn indicators resulted in directional steering corrections. The Arduino executed these commands in real time by modulating PWM signals and motor direction pins, enabling precise movement control. This communication pathway established a closed-loop control system in which environmental perception directly influenced actuation. As a result, the robot continuously sensed, processed, and responded to its surroundings, forming an integrated perception–decision–action cycle that constitutes the foundation of autonomous navigation that is seen across various different industries and car makes today.

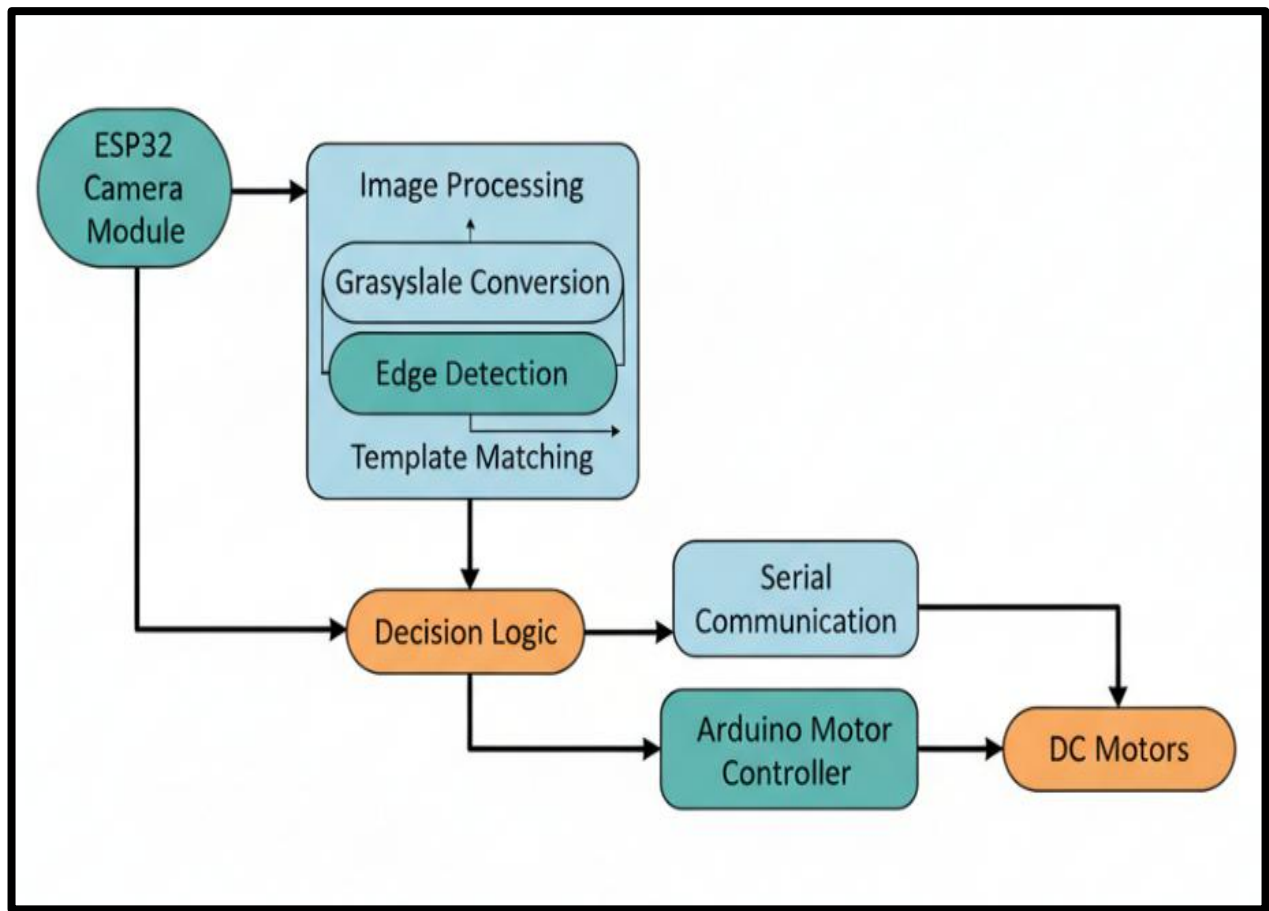


Figure 3: System Architecture Diagram

RESULTS

The autonomous navigation system demonstrated consistent functionality across repeated indoor test trials using camera-based perception and serial motor control. During operation, the ESP32 camera continuously captured forward-facing grayscale frames, which were successfully processed in real time to extract navigational features. The system was able to detect predefined visual cues, including directional arrows and stop signs, and translate those detections into appropriate motor commands through the Arduino-controlled drivetrain. Across structured test environments, the robot reliably maintained forward motion along visually defined paths and executed turns when directional cues were detected. High-contrast printed symbols placed at moderate distances were recognized with high consistency, resulting in correct behavioral responses such as stopping, turning, or continuing forward. When objects were centered within the camera's field of view and lighting conditions were stable, detection latency remained low and navigation behavior appeared smooth and repeatable. Performance degradation was observed under less controlled conditions. Variations in ambient lighting, particularly shadows and glare from reflective surfaces, reduced edge clarity and occasionally caused template mismatches. In these cases, the robot either delayed its response or defaulted to forward motion until sufficient visual confidence was restored. These behaviors highlight the sensitivity of template-based recognition to environmental variability. Overall, the results demonstrate that a low-cost, camera-only autonomous system can achieve functional navigation in structured environments. While not

optimized for unstructured or outdoor use, the system successfully integrates perception, processing, and actuation in real time. These outcomes validate the feasibility of using simplified image processing and embedded control logic to replicate foundational principles of autonomous vehicle navigation at a small scale.

Experimental Setup and Testing Environment

The experimental testing environment used to evaluate the proposed low-cost ESP32–Arduino autonomous navigation system is shown in **Figure 4**. A flat indoor surface was prepared with clearly marked START and END regions using adhesive tape to define the robot's navigation boundaries. Small wooden blocks were distributed throughout the testing area to simulate real-world obstacles in non-uniform positions, requiring the robot to perform dynamic obstacle avoidance. In addition to physical obstacles, traffic instruction signs, including stop, left-turn, and right-turn signs, were mounted on elevated wooden blocks to ensure proper visibility within the ESP32 camera's field of view. This elevation allowed the onboard image processing and computer vision system to reliably detect and classify visual cues. During operation, the robot autonomously navigated from the START position toward the END boundary while simultaneously performing ultrasonic-based obstacle avoidance and executing neural network–based visual decision-making behaviors, such as temporarily stopping at a stop sign and turning according to directional signs. This controlled setup enabled systematic evaluation of both sensor-based navigation and vision-based recognition within a unified testing environment.



Figure 4: Experimental Testing Arena for Autonomous Navigation

DISCUSSION

Interpretation of Results

The results confirm that computer vision-based autonomous navigation can be achieved using accessible hardware and relatively simple algorithms. The successful detection of directional signs and execution of corresponding motor commands demonstrate effective integration between perception, processing, and actuation. This mirrors core components of commercial autonomous vehicle systems, albeit at a significantly reduced scale and complexity.

Errors and Mitigation

During the implementation of autonomous navigation on the Freenove 4WD robot using the ESP32 and Raspberry Pi, several sources of error were observed. These errors impacted the robot's path accuracy, obstacle avoidance, and object recognition. Understanding these challenges provides insight into practical limitations of low-cost robotics systems and offers guidance for students seeking to replicate or improve upon the project. The ESP32 board occasionally experienced resets due to voltage fluctuations or overheating, temporarily halting navigation. Camera modules sometimes dropped frames or produced blurred images, affecting detection of obstacles and path lines. Motors exhibited slight torque variability, causing drift, and wheels occasionally slipped on smooth or uneven surfaces. Environmental factors such as lighting changes, reflective surfaces, and uneven floors further impacted sensor reliability and navigation accuracy.

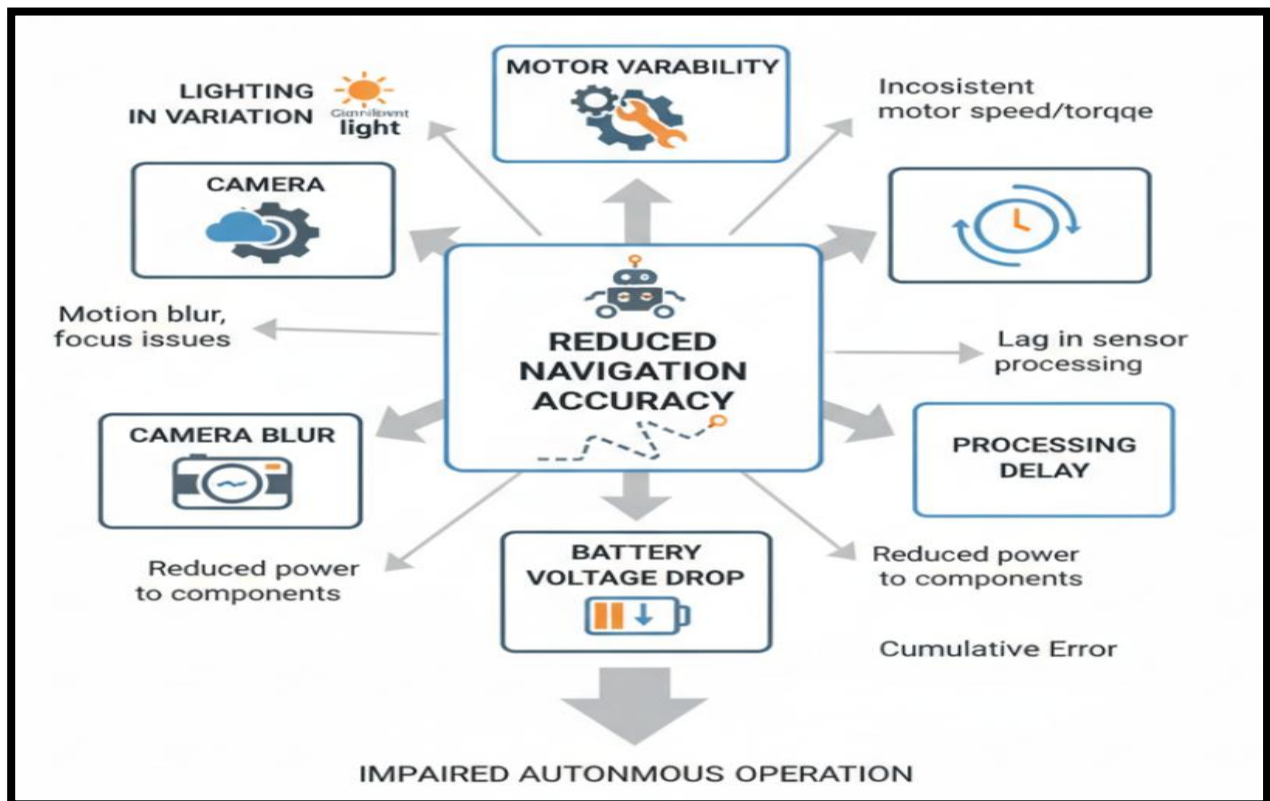


Figure 5: Error Impact Visualization

Processing latency in real-time image analysis slowed obstacle responses, while ultrasonic and infrared sensors sometimes misread distances. Machine learning models misclassified objects under poor lighting or unusual angles, and edge detection occasionally generated false positives on textured surfaces. Navigation errors accumulated over long paths, leading to path drift, and sharp turns were sometimes under- or overshoot due to motor differences. Obstacle avoidance occasionally failed in complex layouts, trapping the robot temporarily. Sensor misalignment, differences in wheel diameter, and motor calibration inconsistencies introduced systematic detection and movement errors. Battery voltage drops further reduced motor performance, slowing reaction times and affecting navigation precision.

Error Type	Description	Mitigation / Tip
ESP32 resets	Board randomly reset due to voltage fluctuations or heat	Use stable power supply; avoid overheating
Camera drops/blurry frames	Missing or blurred frames affected obstacle and path detection	Secure connections; ensure sufficient power
Motor variability	Torque differences caused drift or uneven turns	Calibrate motor speeds; test straight paths
Wheel slippage	Traction loss on smooth or uneven surfaces	Use high-friction surfaces or rubber grips
Processing latency	Delays in real-time image processing slowed obstacle responses	Reduce frame resolution; process fewer frames per second
Sensor inaccuracies	Ultrasonic/IR sensors misread distances, causing collisions	Adjust thresholds; avoid reflective surfaces
Object recognition errors	ESP32 camera misclassified signs or obstacles under poor lighting or angles	Train on varied lighting conditions; use consistent illumination
Edge detection noise	False lines on textured surfaces affected lane detection	Tune thresholds; apply smoothing filters
Path drift	Small steering errors accumulated over time	Recalibrate frequently; implement line-follow correction loops
Turning errors	Sharp turns were under- or overshoot due to motor differences	Adjust turn angles in code based on motor tests
Obstacle avoidance failure	Robot got trapped in complex layouts	Map test areas; clear obstacles; combine multiple sensors
Lighting changes	Ambient light variations reduced camera accuracy	Use consistent lighting or add LED illumination
Surface irregularities	Uneven or reflective floors reduced traction and sensor accuracy	Use flat, non-reflective surfaces
Sensor misalignment	Misaligned cameras or distance sensors caused systematic detection errors	Carefully align sensors; verify before each trial

Battery voltage drop	Reduced motor speed and delayed response as batteries discharged	Use fully charged batteries; monitor voltage
----------------------	--	--

Table 1: Project Errors Log with Listed Mitigation Tips

Limitations and Educational Implications

While effective for instructional purposes, the system is limited by its reliance on template-based recognition and camera-only perception. The absence of sensor fusion and advanced machine learning restricts robustness in unstructured environments. However, these limitations enhance the project’s educational value by making system behavior transparent and easier to analyze for students learning autonomous systems.

CONCLUSION

This project successfully demonstrates that autonomous navigation can be achieved using accessible hardware, simplified image processing techniques, and embedded control systems. Observed limitations, including sensitivity to lighting conditions, motor variability, and power stability, highlight common challenges faced in embedded autonomous systems. Environmental lighting fluctuations affected image contrast and object detection consistency, while variations in motor torque and wheel traction occasionally produced slight deviations in directional accuracy. Power instability under sustained load further demonstrated how hardware constraints can influence computational performance and actuator reliability. However, these limitations also served as opportunities for iterative improvement and system refinement. Mitigation strategies such as motor calibration, lighting control, voltage monitoring, and processing optimization improved reliability and reinforced the importance of system-level engineering considerations. Repeated testing cycles allowed for incremental tuning of PWM motor outputs and refinement of detection thresholds, ultimately improving system responsiveness and behavioral consistency.

In conclusion, this research demonstrates that low-cost robotics platforms can effectively model the fundamental concepts of autonomous navigation. By integrating perception and decision-making within a distributed ESP32–Arduino framework, the project successfully replicated core principles found in larger-scale autonomous systems. The project provides a scalable foundation for future enhancements, including adaptive thresholding, machine learning classifiers, sensor fusion, or real-time environmental mapping. Additional improvements could incorporate lightweight convolutional neural networks optimized through quantization techniques, enhanced power regulation systems, or improved communication protocols for reduced latency. Overall, beyond technical outcomes, this work illustrates how complex autonomous systems can be broken down into manageable components, making advanced engineering concepts accessible and cost-effective to student researchers and educators alike.

REFERENCES

- Banbury, C. R., Zhou, C., Fedorov, I., Navarro, R., Thakker, U., Gope, D., & Reddi, V. J. (2021). *Benchmarking TinyML systems: Challenges and direction*. arXiv. <https://arxiv.org/abs/2003.04821>
- Bradski, G. (2000). *The OpenCV library*. Dr. Dobb's Journal of Software Tools. <https://opencv.org/>
- David, R., Duke, J., Jain, A., Reddi, V. J., Jeffries, N., Li, J., & Howard, A. (2021). *TensorFlow Lite Micro: Embedded machine learning for TinyML systems*. *Proceedings of Machine Learning and Systems*, 3, 800–811.
- Edge Impulse. (n.d.). *Edge Impulse documentation*. <https://docs.edgeimpulse.com/>
- Freenove. (n.d.). *Freenove 4WD car kit for Raspberry Pi*. <https://docs.freenove.com/projects/fnk0050/en/latest/>
- Han, S., Mao, H., & Dally, W. J. (2016). *Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding*. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1510.00149>
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., & Adam, H. (2017). *MobileNets: Efficient convolutional neural networks for mobile vision applications*. arXiv. <https://arxiv.org/abs/1704.04861>
- Kahn, G., Villafior, A., Ding, B., Abbeel, P., & Levine, S. (2020). *BADGR: An autonomous self-supervised learning-based navigation system*. arXiv. <https://arxiv.org/abs/2002.05700>
- Patil, S. S., et al. (2020). *Design and implementation of a vision-based autonomous robot using Raspberry Pi and OpenCV*. *International Research Journal of Engineering and Technology*, 7(7). <https://www.irjet.net/archives/V7/i7/IRJET-V7I7152.pdf>
- Szeliski, R. (2011). *Computer vision: Algorithms and applications*. Springer.
- Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic robotics*. MIT Press.
- Warden, P., & Situnayake, D. (2019). *TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers*. O'Reilly Media.